

SEUPD@CLEF: Team RAND

Giorgia Amato¹, Nicolas Brentel¹, Alessio Demo¹, Steven Laghetto¹, Francesco Pivotto¹ and Iegor Toporov¹

¹University of Padua, Italy

Abstract

This paper presents our work done by team RAND (University of Padua) on LongEval-Web Retrieval challenge, which investigates the robustness and stability of Web search engines in the context of evolving document collections.

Our team began by analyzing the dataset and applying standard IR techniques to establish baseline performance. We then iteratively refined our approach by focusing on methods that demonstrated improved effectiveness in handling temporal changes across snapshots.

Keywords

LongEval 2025, Information Retrieval, Temporal Evaluation

1. Introduction

This report aims at providing a brief explanation of the Information Retrieval system built for Task 1. LongEval-Web Retrieval. The primary objectives are to assess how search engine performance is affected as the underlying Web data changes over time, and to determine when an information retrieval (IR) system requires updates in response to such changes. The task relies on a dataset includes a series of document and query snapshots that reflect the changing nature of the Web over time. The documents and queries were collected monthly, covering the period from June 2022 to February 2023, and are based on real-world user interactions from the Qwant search engine.

The main idea of the RAND group was to design and implement a modular and extensible Information Retrieval (IR) system as part of the CLEF 2025 LongEval lab. The core of the system is a Java-based framework designed to handle the full IR pipeline (including parsing, indexing, searching, and evaluation) while supporting different document formats through a universal parser architecture. To complement this, Python-based components were integrated for experimentation and benchmarking. Special attention was given to exploring the impact of preprocessing strategies (e.g., stopword filtering) and query variations on retrieval effectiveness, allowing the system to adapt to temporally evolving document collections. This dual-language approach provided both performance and flexibility in evaluating IR techniques over time. The paper is organized as follows: Section 2 describes our approach; Section 3 explains our experimental setup; Section 4 discusses our main findings; finally, Section 5 draws some conclusions and outlooks for future work.

2. Methodology

We developed this information retrieval system based on the knowledge gained during the course, particularly by combining the theoretical concepts with the code part presented at the tutor classes. Our starting point was the development of the RANDDocument class and the RANDUniversalParser which could convert different collection documents format (.trec - .json) into RANDDocument object,

“Search Engines”, course at the master degree in “Computer Engineering”, Department of Information Engineering, and at the master degree in “Data Science”, Department of Mathematics “Tullio Levi-Civita”, University of Padua, Italy. Academic Year 2024/2025

✉ giorgia.amato@studenti.unipd.it (G. Amato); nicolas.brentel@studenti.unipd.it (N. Brentel); alessio.demo@studenti.unipd.it (A. Demo); steven.laghetto@studenti.unipd.it (S. Laghetto); francesco.pivotto.1@studenti.unipd.it (F. Pivotto); iegor.toporov@studenti.unipd.it (I. Toporov)



© 2025 Copyright for this paper by its authors.

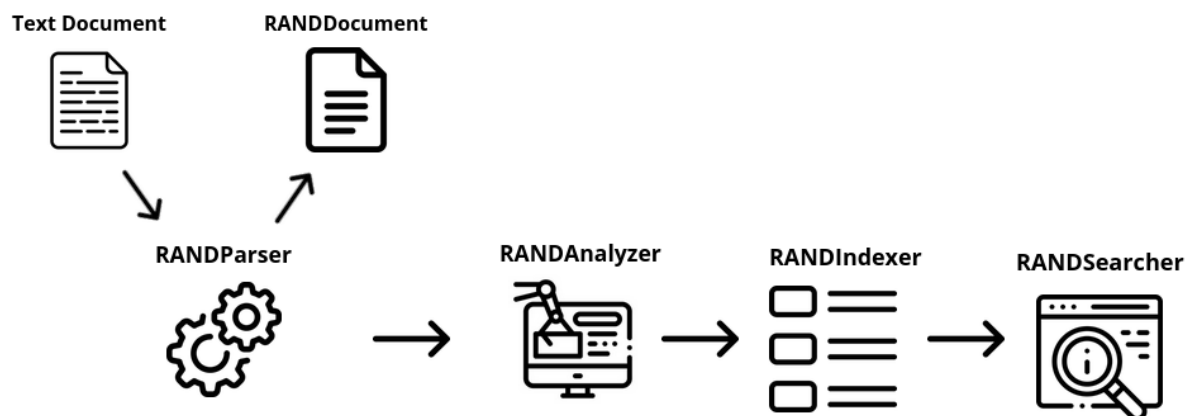
Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

then we developed the RANDAnalyzer class that is able to perform tokenization, stop-list application and stemming operations.

The next step was to develop the Searching system with the goal of performing a topic search on the collections from a query file (topics) and generating a “run” file in TREC format with the results.

The last element of the pipeline is the evaluation phase by developing the Evaluator class that uses the trec_eval tool to derive metrics (retrieved documents, relevant documents, DCG, etc...).

Finally The main class joins the entire pipeline with the help of the FileCollector class that allows to derive files from a folder path (even composed of subfolders).



2.1. Workflow and foundations

2.1.1. Base system

The base pipeline is composed of three main phases: parsing, indexing and searching. For the parsing, the system can accept JSON or TREC files as input, from which it will clean the text by removing unwanted elements like HTML tags, links and special characters. The indexing phase is responsible for converting a collection of text documents into a searchable format using a text indexing framework. The system allows customization of how text is tokenized and compared. The searcher phase is responsible for executing search operations over the index using a set of predefined queries. It can apply advanced text processing techniques, including tokenization and synonym-based query expansion. The system supports configurable parameters and can integrate additional semantic tools to enhance relevance. The final ranked documents are put into a standardized output suitable for evaluation.

2.1.2. Re-ranking

In order to get the most relevant documents to the top, we implemented inside the RANDSearcher class a **re-ranking** mechanism designed to improve the quality of the retrieved documents by enhancing the initial ranking provided by Lucene's scoring (e.g., BM25).

This post-retrieval phase re-evaluates each document's relevance based on a semantic similarity computation between the query and the document content.

After the initial search performed using Lucene, the retrieved documents (stored as `ScoreDoc[]` objects) can be re-ordered based on additional similarity signals obtained from external services. The re-ranking logic adjusts the score of each document based on its semantic closeness to the original query.

2.2. Implementation

In this section we describe the main classes developed for our system. The final execution is delegated to the `Main` class.

2.2.1. RANDDocument

The `RANDDocument` class represents an abstraction of an indexed and searchable document within the system, handling both TREC and JSON file formats.

This class parses the input data and converts it into a format that can be processed by the indexing system.

For compatibility with the Lucene indexing engine, `RANDDocument` includes:

- **IdField:** This field uniquely identifies each document and is stored, but not tokenized, to ensure efficient retrieval.
- **BodyField:** This field contains the main textual content of the document and is tokenized for full-text search.
The body field is stored explicitly to allow re-ranking based on the content's relevance to the query.

Incorporating both TREC and JSON formats into the pipeline allows the system to handle a variety of input formats while maintaining efficient indexing and retrieval processes.

2.2.2. RANDParser

RANDUniversalParser Looking at the collections to be parsed, the need to manage both `json` files and `trec` files is immediately apparent.

For this reason, it was decided to create a `RANDUniversalParser.java` class, which can treat the input file differently depending on its extension:

- If the input file is of type `json`, then it will be passed to the class `RANDJsonParser.java`.
- If it is of type `trec`, it will be passed to the class `RANDTrecParser.java`.
- If it has an extension other than `json` or `trec`, an error is returned.

RANDJsonParser.java `RANDJsonParser.java` was built on the basis of `WapoParser.java` seen during the tutorships. Since the file to be processed is of type `json`, the *Jackson JSON* library is used to parse the file.

The core of the class is the `next()` method, which allows for scrolling through all uploaded documents, one at a time, applying cleanup (such as removal of emojis, URLs, and HTML tags) on each text before returning it.

RANDTrecParser.java `RANDTrecParser.java` was developed from the class `TipsterParser.java` seen during the lectures. It consists of a very basic parser for `trec` files that identifies the individual documents enclosed by the `<DOC>` and `</DOC>` tags, extracts the document identifier and the text contained between the tags, and removes any emojis, URLs, and HTML tags through the `cleanContent()` method.

2.2.3. RANDAnalyzer

This class represents the analyzer of our Information Retrieval system.

It was developed with the goal of giving us maximum flexibility during the testing phase, making it fully hybrid: it allows analyzing documents using both tokenizers and stemmers for English and for French.

Thanks to the configuration saved in an XML file, it was possible to define the analysis parameters in a

flexible and efficient way.

The system supports three types of tokenizers selectable via the `TokenizerType` parameter.

- `WhitespaceTokenizer`,
- `LetterTokenizer`
- `StandardTokenizer`,

After tokenization, the terms are converted to lowercase using `LowerCaseFilter`, filtered by length using `LengthFilter`, and then subjected to stopwords removal using `StopFilter`.

Then two main configurations were adopted: one for English and one for French.

In the English case, the `EnglishPossessiveFilter` is applied to remove possessive forms, followed by the application of a stemmer, selectable from:

- `EnglishMinimalStemFilter`,
- `PorterStemFilter`,
- `KStemFilter`,
- `SnowballFilter`.

While in the French case, the system applies:

- `ElisionFilter`,
- `FrenchLightStemFilter`,
- `ICUFoldingFilter`.

After several tests, we decided to adopt the configuration for the French language (excluding the `ElisionFilter` because it made performance worse), as the majority of relevant documents are in French and this configuration showed the best performance as can be seen in the graph in section 4.

2.2.4. VocabularyStatistics

To ensure the correctness of the index and facilitate the analysis of all its fields, we initially considered using *Luke* (Lucene Index Toolbox). Given the lack of practicality in its usage, we therefore sought an alternative solution. By reviewing projects from two years ago, we found that a team (GWCA¹) had developed a custom Java class named `VocabularyStatistics`, which suited our needs perfectly. This utility generates a text file that lists all the terms contained in the Lucene index, along with associated frequency statistics.

Specifically, the class includes a method that produces a file named `vocabulary.txt`, which contains all indexed terms sorted in descending order based on their raw frequency. Furthermore, the class automatically appends the most frequent terms to the stoplist (removing duplicates, if present). However, testing showed that the manually curated version of `oracleFrench.txt` was already highly effective, and additional automatic expansions turned out to be counterproductive. Consequently, we decided to retain the original version of `oracleFrench.txt` as the stoplist.

2.2.5. RANDIndexer

Our Indexer is responsible for:

- Creating a Lucene index from a collection of parsed documents produced by the Parser.
- Avoiding duplicates and invalid entries during the indexing process.
- Allowing customization via analyzers and similarity metrics (e.g. BM25).

In details the Indexer is composed by the following components:

¹GWCA Team's repository: <https://bitbucket.org/upd-dei-stud-prj/seupd2223-gwca/src/master/>.

- **The constructor** that receives as parameters:
 - The Analyzer that tokenizes and process the text of the documents.
 - The Similarity that determines ranking model.
 - A integer that describes how much RAM is used to buffering documents before writing it on the disk.
 - The file path where the index is stored.
 - The parser that is responsible for the documents parsing.
- **index() method** that checks the validity of input parameters (skips null documents or those without a valid ID).
 In addition this method converts each RANDDocument to a Lucene Document thanks to the Parser's method 'toLuceneDocument()' and adds it to the index; and finally commits and closes the index, printing a report regarding the indexed documents, the skipped documents and the duplicates.

2.2.6. RANDSearcher

The RANDSearcher is our Lucene-based search component designed to retrieve relevant documents from a pre-built index using queries in TREC format.

It operates by reading a set of topics from a tab-separated file, where each line contains a query identifier and its description.

These queries are analyzed using a custom RANDAnalyzer, which ensures consistency with the analyzer used during indexing. For each query, the system builds a BooleanQuery targeting the Body field of the indexed documents.

Tokens extracted from the query description are converted into Lucene TermQuery objects. If synonym-based query expansion is enabled via configuration, synonyms are retrieved using the WolfManager class, which provides a list of synonyms of a given word based on the WOLF dataset², and a SynonymQuery is built for each token with weighted contributions from synonyms.

After forming the initial query, the IndexSearcher executes the search with a configurable number of maximum documents to retrieve.

Optionally, if the top result's score is below a given threshold, a re-ranking phase is triggered. In this phase, either a deep learning model or a large language model (LLM) is used to re-score the retrieved documents based on semantic similarity between the query and the document bodies.

The final ranked list is written to a run file in standard TREC format, avoiding duplicate document IDs per query.

This output can be used for evaluation with tools such as trec_eval.

The search process is controlled via an XML-based configuration (SearcherParams), and all relevant components (analyzer, similarity model, expansion strategies, and re-ranking method) are easily configurable, allowing for flexible experimentation.

Also, as said previously, the RANDSearcher class is in charge of executing the re-ranking of the retrieved documents.

A briefer description of the whole process is provided below:

1. **Initial retrieval:** Lucene performs a standard ranked retrieval using a Boolean query built from tokenized query terms, optionally enriched with synonyms from Wolf. This first passage uses BM25 similarity.
2. **Optional re-ranking:** In case the list of the top ranked documents doesn't satisfy a given threshold, the similarity between the document body and the query is computed using one of the following two approaches, based on a configurable parameter inside the SearcherParams class:
 - DL (Deep Learning)
 - LLM (Large Language Model)

²WOLF, Free Wordnet for French. Synset available at https://almanach.inria.fr/software_and_resources/WOLF-en.html.

3. **Score adjustment:** Once the new score is computed, the it is scaled and added to the original Lucene score with: `sd[i].score += (float) (5 * scores.get(i))` This alters the position of the document in the result set.
4. **Final output:** The updated scores are used when writing the run file in TREC format. This allows for improved evaluation using external tools such as `trec_eval`.

As mentioned before, this feature is controlled via the `SearcherParams` object, specifically the field `getReRanker()`, which can be set to either DL, LLM, or disabled with `None`. When enabled, re-ranking can help surface more semantically relevant documents that traditional keyword-based retrieval might overlook, especially in cases of vocabulary mismatch or when query understanding is critical.

3. Experimental Setup

3.1. Collection

For our project, we utilized the **LongEval 2025 Web Retrieval Train Collection**, curated by TU Wien in collaboration with Qwant.

This dataset is designed to support research in temporal information retrieval and the evaluation of long-term relevance persistence.

Dataset Composition:

- **Queries:** 9,000 user-issued queries focused on trending topics, extracted from Qwant’s search logs between June 2022 and February 2023.
- **Documents:** Approximately 18 million web documents retrieved in response to those queries, including both clicked results and randomly sampled content from Qwant’s index.
- **Relevance Judgments:** Graded relevance labels derived from a click model, reflecting user interaction and engagement patterns.

Data Characteristics:

- The dataset includes original French versions of both queries and documents, adding a multilingual dimension to retrieval tasks.
- Documents are stored in JSON format, with metadata such as document ID, URL, and timestamp, enabling structured parsing and temporal analysis.
- The temporal coverage of the data captures language evolution and topical drift, challenging systems to maintain performance over time.

This collection is used as the official training set for the LongEval 2025 Information Retrieval Lab at CLEF and provides a robust benchmark for evaluating retrieval systems in dynamic and realistic web search scenarios.

3.2. Evaluation Measures

We evaluate the performance of our information retrieval system using three key metrics: **Mean Average Precision (MAP)**, **Normalized Discounted Cumulative Gain (nDCG)**, and **Normalized Discounted Cumulative Gain at rank 10 (nDCG@10)**.

- **MAP (Mean Average Precision):** MAP provides an overall measure of the retrieval ability of the system to return relevant documents. It calculates the average precision for each query and then averages these values across all queries. This metric is useful for assessing how well the system ranks relevant documents throughout the entire result list.
- **nDCG (Normalized Discounted Cumulative Gain):** nDCG evaluates the ranking quality of the top documents by giving more weight to relevant documents at the top of the result list. It is particularly valuable for assessing systems where the relevance of documents in higher ranks is more significant.
- **nDCG@10 (Normalized Discounted Cumulative Gain at rank 10):** This variation of nDCG emphasizes the relevance of the top 10 documents. It applies a logarithmic discount to the relevance of documents at lower ranks, giving more importance to those ranked near the top.

We used the official `trec_eval` tool to calculate both MAP and nDCG metrics based on our system outputs and the provided `qrels` file.

The results will be discussed in detail in Section 4.2.

3.3. Git

To develop the project, our group made use extensively of git in order to collaborate and organize the files.

The full source code is available on our BitBucket Repository.³

³RAND bitbucket repository: <https://bitbucket.org/upd-dei-stud-prj/seupd2425-rand/src/master/>.

3.4. Hardware

The used hardware involved different systems to manage the heavy workload on the machines more time-efficiently.

These were the machines that were used:

Machine 1		Machine 2	
CPU	11th Gen Intel Core i7-1165G7	CPU	AMD Ryzen 5 3600
RAM	8GB 3200 MHz	RAM	16GB 3600 MHz
OS	Windows 11	OS	Windows 11
GPU	Intel Iris Xe Graphics	GPU	NVIDIA GTX 1650 Super

Machine 3		Machine 4	
CPU	12th Gen Intel Core i5-12400F	CPU	AMD Ryzen 3 5300U
RAM	16GB 3600 MHz	RAM	8GB 3200 MHz
OS	Windows 11	OS	Ubuntu
GPU	AMD 6650XT	GPU	Integrated Radeon Graphics

Machine 5	
CPU	8th Gen Intel Core i5-8250U
RAM	16GB 2400 MHz
OS	Windows 10
GPU	Intel UHD Graphics 620

Table 1

Hardware specification of the four machines used in the experiments

4. Results and Discussion

In this section, we describe (4.1) and discuss the runs-and their results (4.2), evaluated with the `trec_eval` tool-produced by the different configurations of our system.

4.1. Runs description

Here we describe the configurations of our system, with a summary of tokenizer, filters, stoplist, stemmer, and the parameters of our RANDSearcher (2.2.6).

4.1.1. seupd2425-rand-nofilters

For the first run, we decided to use the standard configuration of our RANDAnalyzer (2.2.3, WhitespaceTokenizer, LowerCaseFilter, LengthFilter, which will be used in every run reported here) without any stoplist or stemmer.

Tokenizer: Whitespace

Filters: LowerCase, Length

Stoplist: None

Stemmer: None

Searcher parameters: None

4.1.2. seupd2425-rand-englishFilter

For the second run, we focused on evaluating the performance of our IR system specifically on English-language documents, as the CLEF dataset includes a subset of documents written in English. In this configuration, we retained the standard setup of our RANDAnalyzer (see Section 2.2.3, which includes the WhitespaceTokenizer, LowerCaseFilter, and LengthFilter, used in all reported runs), and extended it with English-specific processing.

This involved enabling a filter to remove English possessive suffixes (e.g., 's), applying a stopwords list tailored to English, and using the Porter stemmer to reduce words to their root forms. These additions were intended to better normalize and filter the English texts, thus improving retrieval effectiveness for this subset of the collection.

Tokenizer: Whitespace
Filters: LowerCase, Length, English Possessive suffixes
Stoplist: Azure Stoplist of Igor Brigadir⁴
Stemmer: Porter
Searcher parameters: None

4.1.3. seupd2425-rand-queryLength

Following the disappointing performance observed when applying English-specific tokenization and stemming techniques, we shifted our focus to documents written in French, which also constitute a significant portion of the CLEF dataset. For this run, we adapted our RANDAnalyzer configuration to better suit the linguistic characteristics of French. While maintaining the base components (WhitespaceTokenizer, LowerCaseFilter, and LengthFilter), we replaced the English-specific modules with their French counterparts.

In particular, we employed a minimal French stemmer to reduce inflectional variation, and we used the Oracle French stoplist to eliminate common non-informative French terms. Furthermore, in response to the observation that many queries consisted of only a single token, we relaxed our query-document matching constraint: instead of requiring at least two matching terms between query and document (as done in previous runs), we allowed matches with only one shared term. This adjustment aimed to improve retrieval for short queries that would otherwise have been excluded by stricter matching thresholds.

Tokenizer: Whitespace
Filters: LowerCase, Length
Stoplist: Oracle French Stoplist⁵
Stemmer: FrenchLight
Searcher parameters: Minimum query-document match threshold set to 1 term

4.1.4. seupd2425-rand-frenchFilter

However, this relaxation resulted in a noticeable degradation in retrieval performance. Although including short, single-token queries increased coverage, these queries often failed to retrieve a sufficient number of relevant documents. As a result, they significantly reduced the system's overall effectiveness, as reflected by a lower NDCG score. The low quality of the results suggested that the looser matching constraint prioritized recall at the cost of precision, making retrieval noisier and less reliable.

Consequently, from this run onward, we decided to discard results from queries that did not match at

⁴English stoplist compiled by Igor Brigadir, available at <https://github.com/igorbrigadir/stopwords/blob/master/en/azure.txt>.

⁵French stoplist compiled by Igor Brigadir (Oracle version), available at <https://github.com/igorbrigadir/stopwords/blob/master/fr/oracle.txt>.

least two terms with the documents. This stricter constraint was reinstated to ensure a higher quality of retrieval, favoring precision and relevance over mere query inclusivity.

Tokenizer: Whitespace

Filters: LowerCase, Length

Stoplist: Oracle French Stoplist⁶

Stemmer: FrenchLight

Searcher parameters: Minimum query-document match threshold restored to 2 terms

4.1.5. seupd2425-rand-DL

This run for reRanking purposes aims to improve the precision of relevant documents placing them in high-rank positions.

In these runs, we also used the standard configuration of the Analyzer as in the previous one.

Tokenizer: Whitespace

Filters: LowerCase, Length

Stoplist: Oracle French Stoplist

Stemmer: FrenchLight

Searcher parameters: Deep Learning reRanker (DL)

4.1.6. seupd2425-rand-synonyms

In this run the synonym parameter is applied; in the other hand synonyms weight parameter assumes different values.

High values improve recall (more documents found, but at risk of less accuracy).

Low values improve precision (less 'noise' but you may miss relevant documents that use only synonyms).

Standard Analyzer's configuration is used.

Tokenizer: Whitespace

Filters: LowerCase, Length

Stoplist: Oracle French Stoplist

Stemmer: FrenchLight

Searcher parameters: Synonyms parameter is set to 1, and three values are applied to the synonyms weight parameter: 0.5, 0.8 and 1.

This is useful to balance the importance of synonyms against the original terms in document matching. No reRanker applied.

4.1.7. seupd2425-rand-elisionFilter

After the previous runs, we noticed that our system performs better with the synonym weight set to 0.5 (Fig.2). For this reason, we decided to keep this value also for another run configuration, where we applied an ElisionFilter to remove elisions, frequently present in the French language (e.g. articles).

Tokenizer: Whitespace

Filters: LowerCase, Length, Elision

Stoplist: Oracle French Stoplist

Stemmer: FrenchLight

Searcher parameters: synonyms with weight=0.5

⁶French stoplist compiled by Igor Brigadir (Oracle version), available at <https://github.com/igorbrigadir/stopwords/blob/master/fr/oracle.txt>.

4.1.8. seupd2425-rand-ICU

For this run, we started from the standard French configuration (4.1.4) and introduced the Lucene ICUFoldingFilter that applies the search term folding to Unicode text, with accent removal and case folding, among others. This turned out to be useful, as the French language contains many words with diacritical marks (accents), which can affect search and comparison. In fact, this filter improves the performance of our system: we can see in Fig.1 that this configuration obtained the highest value of nDCG among the tested configurations.

Tokenizer: Whitespace
Filters: LowerCase, Length, ICUFolding
Stoplist: Oracle French Stoplist
Stemmer: FrenchLight
Searcher parameters: None

4.2. Results

The table below presents the final averaged evaluation metrics for each system configuration (run). We report three key metrics: **nDCG**, **nDCG@10**, and **MAP**. These metrics provide a comprehensive view of the retrieval performance across different strategies applied during the experiments, including language filtering, synonym expansion, and reranking.

- **nDCG (Normalized Discounted Cumulative Gain)** measures the overall ranking quality by emphasizing the placement of relevant documents higher in the ranked list.
- **nDCG@10** focuses specifically on the quality of the top 10 retrieved documents, aligning with typical user behavior in web search.
- **MAP (Mean Average Precision)** captures the average precision of relevant documents across all queries, providing an overall measure of relevance and ranking accuracy.

The reported values represent the average scores obtained over nine monthly runs. These results help to highlight the relative strengths of each configuration and guide the selection of the most effective retrieval strategy.

As discussed previously, the ICU configuration (4.1.8) was revealed to have the best performance in terms of each of the three evaluation measures considered for the comparison.

The FrenchFilter (4.1.4) runs turned out to have the second-best nDCG, while the re-ranking mechanism adopted in the DL configuration (4.1.5) leads to higher nDCG@10 and MAP values, as expected.

Table 2

Final averaged evaluation metrics (nDCG, nDCG@10, MAP) for each run

Run Name	nDCG	nDCG@10	MAP
seupd2425-rand-nofilters	0.1423	0.0951	0.0804
seupd2425-rand-frenchfilter	0.1638	0.1071	0.0950
seupd2425-rand-englishfilter	0.1223	0.0803	0.0679
seupd2425-rand-elisionfilter	0.1482	0.1004	0.0851
seupd2425-rand-DL	0.1401	0.1086	0.0972
seupd2425-rand-synonyms0.5	0.1475	0.1001	0.0849
seupd2425-rand-synonyms0.8	0.1393	0.0934	0.0789
seupd2425-rand-synonyms1	0.1318	0.0965	0.0762
seupd2425-rand-ICU	0.1707	0.1170	0.0995

The chart (Fig.1) shows the nDCG trend over time, evaluating the effectiveness of different parameter configurations in search runs; in particular, for re-ranking we retrieved a maximum of 100 documents

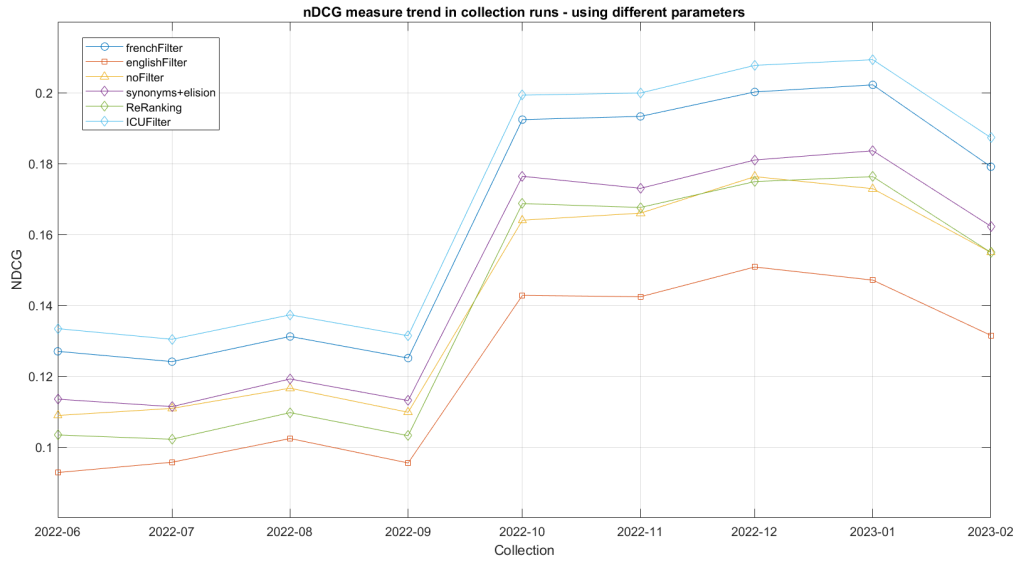


Figure 1: Comparison of nDCGs of different system configurations over the collection months.

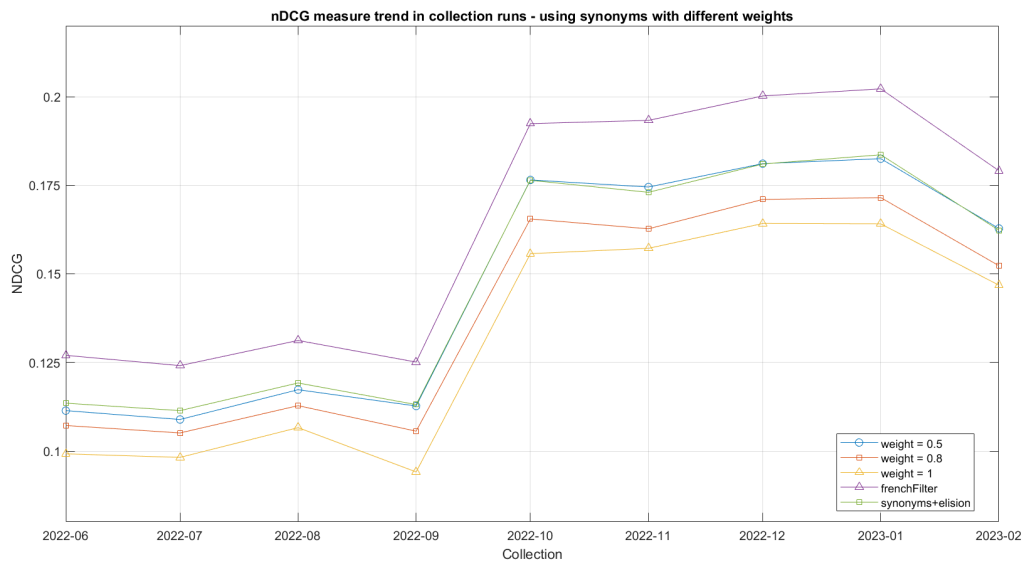


Figure 2: Comparison of system configurations using various synonym weights.

per query, due to the execution time being too long to retrieve 1000 documents per query, this is also why the nDCG parameter is particularly low.

In particular, we can deduce the following trends from the graph:

- All configurations show a clear improvement starting around October 2022, probably due to the higher quality of the collections (e.g., better structured, less noisy), making it easier to return relevant documents.
- The most notable gains are seen with French-specific filters (4.1.4). Probably the reason is that the queries and documents are matched in the same language or tokenization style, reducing noise greatly and improving ranking quality.
- Without filters (4.1.1), irrelevant terms or tokens might be included in both indexing and querying stages, causing mismatches and drops in performance compared to runs with filters

applied.

- reRanking (4.1.5) uses more sophisticated models (e.g., DL or LLM-based) to reorder the top results, improving the placement of highly relevant documents, but due to the fact that we set the parameter `maxDocsRetrieved` to 100, we have a drop in metrics.
- synonyms+elision (4.1.7) offers a moderate boost over `noFilter`, indicating that query expansion helps but is not as impactful as filtering.

The second graph (Fig.2) compares the different configurations of our system using various synonym weights (4.1.6) for query expansion. Runs with `weight = 0.5` achieve better results in terms of `nDCG`, while increasing the weight leads to a drop in performance.

The comparison also includes the runs with the `frenchFilter` (4.1.4) and the `elisionFilter` (4.1.7), the latter using query expansion with a synonym weight of 0.5. Across all months in the collection, query expansion results in a degradation of system performance when compared to the `frenchFilter` configuration. The introduction of the `ElisionFilter` does not cause significant changes in the `nDCG` values compared to the `weight = 0.5` configuration, but performs slightly better on average in terms of `nDCG`, `nDCG@10` and `MAP`.

5. Conclusions and Future Work

5.1. Conclusions

This work presented a modular information retrieval system built on top of the Lucene framework, enhanced with custom query formulation strategies and optional query expansion through Wolf, using optionally deep learning or large language models for the re-ranking phase.

The system demonstrated flexibility in handling TREC and JSON topics and produced standardized run files suitable for evaluation.

Initial experimentation were based on few filters configuration, then French-dedicated stoplist and stemmer were introduced to improve our system performance.

The implementation of a Deep Learning model for reranking seemed to be promising in terms of `nDCG@10`, but its high computational cost forced us to retrieve a maximum of 100 documents per query, and this led to a drop of `nDCG`.

5.2. Future Work

Future developments will focus on integrating deep learning models and large language models (LLMs) for more effective and efficient query rewriting and document re-ranking.

These enhancements aim to boost semantic understanding and retrieval precision.

Further work will also explore improved query expansion with synonyms, incorporating relevance feedback mechanisms, a better management of short queries, and refining scoring models to better capture user intent.

6. Group Members Contribution

The project was collaboratively developed by all group members.

Below is a summary of the main contributions made by each participant, based on the Git commit history:

Giorgia Amato Contributed to the design and implementation of the `RANDDocument` class and its integration with the Lucene indexing framework, including utility methods such as `idField()` and `bodyField()`. She also developed the `Evaluator` component used to compute retrieval metrics via `trec_eval`. In the report, she authored the `Collection`, `Evaluation Measures`, and `Results` sections and participated in multiple rounds of revision.

Nicolas Brentel Focused on query expansion and document processing. He implemented the `WolfManager` for managing French synonyms, developed a Python script to clean the `qrels` file, and contributed to the integration of LLMs and DL models through an HTTP-based helper and Python server. He also made adjustments to document identifiers in run files and contributed to the `AnalyzerParams` configuration. Runs and evaluation measures of `noFilters`, synonyms, `elisionFilter`, ICU configurations. Description of `noFilters`, `elisionFilter`, ICU configurations, part of results analysis and conclusions in the dedicated LaTeX sections. Contribution to matlab script for graphs.

Alessio Demo Developed the core search engine pipeline and the `Main` class, implemented the `Indexer`, improved file retrieval for `.json` and `.trec` documents via the `FileCollector`, and added support for different query formats. He also contributed to Lucene integration, fixed issues in the `Searcher`, and documented the code extensively using Javadoc. He also helped significantly with the LaTeX report, particularly the `Indexer`, `Methodology` and `Results` sections and produced `FrenchFilter`, `ReRanking`, synonyms and ICU runs and contributed to the production of the matlab code.

Steven Laghetto Developed the universal parser (`RANDUniversalParser`) for both `.json` and `.trec` documents, and implemented specific parsers (`RANDJsonParser`, `RANDTrecParser`). He created the `ResourceUtil` class. He found and integrated the `ElisionFilter` and the `ICUFoldingFilter`. He discovered and integrated the `VocabularyStatistics` class. He worked on the integration of Javadoc in most java classes. He contributed in compiling the LaTeX file.

Francesco Pivotto Contributed to the development of `RANDAnalyzer`, including attribute definition, constructor implementation, and resolving issues in the `Indexer`. Actively participated in selecting and running experiments like `englishFilter`, `queryLength`, and `elisionFilter`, optimizing parameters and synonym weights. Contributed to error analysis and corrections in the re-ranking pipeline, and implemented `MinimumNumberShouldMatch` in `RANDSearcher` for single-token queries. Also standardized the run folder structure and documented methodologies for `RANDAnalyzer` and `RANDDocument` in the report.

Igor Toporov Contributed significantly to both the development and documentation of the project. They implemented key components such as the `RANDSearcher` and enhanced the `RANDAnalyzer` with fundamental methods. Additionally, he introduced utility classes to support configuration and parameter handling. On the documentation side, he updated important LaTeX sections including the methodology, results and conclusions, and added Javadoc comments to core classes, helping to improve code clarity and maintainability.

The project was a result of a highly collaborative effort, with members often reviewing and building upon each other's work. All members contributed to debugging, refining features, and writing documentation.